

PetConnect

Plan de Pruebas — Guía Completa

Sprint 6 · Versión 2.0 · Junio 2026

Proyecto	PetConnect (Backend + Frontend PWA)
Repositorios	Darkolnacio/PetConnectBackend · Darkolnacio/PetConnect
Fecha	Junio 2026
Total casos	105 casos de prueba – 188 test automáticos

Integrantes:

Anais Llancapan
Edgar Morales
Catalina Rosales
Diego Sotelo

1. Introducción.....	3
1.1. Propósito y Objetivos del Aseguramiento de Calidad (Sprint 6)	3
1.2. Alcance y Límites de la Auditoría Técnica (Backend, Frontend y PWA)	3
1.3. Origen de los Requerimientos: Backlog de Jira	3
1.4. Criterios de Aceptación del Negocio (Mapeo de las 31 Historias de Usuario)	4
2. METODOLOGÍA Y ARQUITECTURA DE PRUEBAS	5
2.1. Tecnologías, Frameworks y Comandos Reales de Ejecución.....	5
2.2. Configuración de Ambientes de Prueba Aislados	5
2.3. Guía de Ejecución Técnica (Comandos de Consola y Prerrequisitos)	6
2.4. Ciclo de Vida de Integración Continuada (CI/CD) en GitHub Actions	6
3. RESULTADOS CUANTITATIVOS Y CERTIFICACIÓN DE ENTORNOS.....	7
3.1. Cuadro Resumen de la Cobertura Técnica Automatizada.....	7
3.2. Auditoría y Conteo de Tests Automatizados Exitosos (150 Jest / 38 Vitest)	7
3.3. Certificación del Entorno: Análisis del Flujo Smoke (SMK-001 → SMK-015).....	8
4. MATRIZ DETALLADA DE TRAZABILIDAD TÉCNICA PC-*	10
4.1. Estructura de Diseño y Campos de las Tablas de Casos de Prueba	10
4.2. Mapeo Técnico de Casos Implementados por Módulos del Sistema	10
5. ANÁLISIS DE BRECHAS Y TRAZABILIDAD EXTERNA	17
5.1. Evaluación del Porcentaje Real de Cobertura de la Matriz (Cálculo del ~59% Real)	17
5.2. Inventario de Casos Críticos de Prioridad Alta Sin Automatizar	17
5.3. Discrepancias entre Especificaciones del Plan Teórico y Código Fuente	17
5.4. Estado de la Trazabilidad y Sincronización con Herramientas (Jira Test / Xray)	18
6. GESTIÓN DE RIESGOS EN EL PROCESO DE PRUEBAS.....	19
6.1. Criterios de Entrada para el Ciclo de QA	19
6.2. Criterios de Out / Salida para la Aprobación del Release	19
6.3. Matriz de Riesgos Técnicos e Identificación de Mitigaciones	19
7. CONCLUSIONES	21
7.1. Dictamen de Calidad sobre el Sprint 6.....	21
7.2. Lecciones Aprendidas de la Estrategia de Testing	21
7.3. Plan de Acción y Próximos Pasos de QA para Futuros Sprints	21

1. Introducción

1.1. Propósito y Objetivos del Aseguramiento de Calidad (Sprint 6)

El propósito de este informe es certificar el estado del arte en materia de calidad, robustez y estabilidad del sistema PetConnect al cierre del Sprint 6. En el contexto del desarrollo de software moderno, este documento se constituye como la única fuente de verdad técnica, contrastando la planeación teórica del aseguramiento de calidad frente a la realidad empírica del código fuente. Los objetivos principales comprenden la validación estricta de las reglas de negocio, la detección de regresiones en las capas lógicas y la cuantificación de las brechas de automatización para mitigar riesgos antes del despliegue en ambientes productivos.

1.2. Alcance y Límites de la Auditoría Técnica (Backend, Frontend y PWA)

La auditoría técnica abarca la totalidad del ecosistema de software de PetConnect distribuido en sus dos repositorios oficiales: el lado del servidor (PetConnectBackend) y la interfaz de usuario en formato de Aplicación Web Progresiva (PetConnect). El alcance incluye:

- **Capa Backend:** Controladores, servicios de persistencia, utilidades criptográficas y enrutamiento.
- **Capa Frontend:** Manejo de estado global, renderizado de componentes críticos, utilidades de conversión horaria y persistencia local.
- **Capa PWA/E2E:** Estado fuera de línea de la aplicación y flujos de usuario simulados.

Límites: Quedan fuera del alcance de este periodo las pruebas de estrés, pruebas de inyección de vulnerabilidades avanzadas (Pentesting) y la validación en dispositivos móviles nativos, concentrándose la estrategia en pruebas funcionales, unitarias y de integración.

1.3. Origen de los Requerimientos: Backlog de Jira

La base documental que rige el comportamiento esperado del sistema proviene del Backlog oficial del proyecto alojado en Atlassian Jira, extraído sistemáticamente en el archivo de control PetConnect_Jira_Backlog.xlsx. De acuerdo con la auditoría de gestión sobre dicha base de datos, el proyecto acumula un histórico consolidado de **222 registros de trabajo (Issues)** que sustentan el ciclo de vida del software. Este volumen se desglosa técnicamente en **7 Épicas estructurales, 33 Historias de Usuario base, 13 Tareas operativas y 169 Subtareas técnicas** asociadas al desarrollo de código y control de calidad. El esfuerzo estimado total de desarrollo y estancamiento representa una carga de **335 Story Points (SP)** distribuidos a lo largo del ciclo de vida evolutivo del software.

1.4. Criterios de Aceptación del Negocio (Mapeo de las 31 Historias de Usuario)

De las historias de usuario identificadas en el backlog, la suite principal del plan de pruebas se concentró en mapear el núcleo del negocio representado desde la **HU-01 hasta la HU-31**. Cada una de estas historias contiene la definición de las condiciones de satisfacción o Criterios de Aceptación (CA) redactados originalmente por el Product Owner (PO).

Para efectos de estructurar la estrategia de pruebas del Sprint 6, los requerimientos de negocio se agruparon de forma cronológica por su madurez y estimación de puntos de historia en el backlog:

- **Fase de Autenticación y Perfiles (Sprint 1):** 15 Historias de Usuario enfocadas en el aislamiento de roles (dueño, proveedor, admin), registro seguro y recuperación de accesos (57 SP acumulados).
- **Fase de Core del Negocio (Sprint 2):** Concentración de esfuerzo en el motor de mapas, geolocalización de servicios y arquitectura de agendamiento/bloqueo de ventanas de tiempo (76 SP acumulados).
- **Fase de Gestión Veterinaria y Clínicas (Sprints 3 al 5):** 16 Historias de Usuario que introducen el manejo de registros médicos cronológicos, carga de adjuntos, generación de reportes clínicos PDF, el chatbot interactivo y el panel de administración centralizado (144 SP acumulados).

2. METODOLOGÍA Y ARQUITECTURA DE PRUEBAS

2.1. Tecnologías, Frameworks y Comandos Reales de Ejecución

La estrategia de QA de PetConnect utiliza un enfoque piramidal multicapa para aislar y validar el comportamiento de los componentes de software de manera independiente o integrada:

- **Capa 1 (Lógica de Servidor):** Implementada con **Jest 29** y **Supertest**. Realiza aserciones directas sobre los ciclos de petición/respuesta HTTP (request-response) simulando llamadas de red sin levantar servidores externos.
- **Capa 2 (Componentes de Interfaz):** Soportada por **Vitest 4** y **React Testing Library**. Evalúa de forma unitaria la respuesta del DOM ante eventos de usuario, cambios de propiedades y mutaciones de contexto.
- **Capa 3 (Validación de Integración de API):** Ejecutada mediante **Newman CLI**, el motor de ejecución en consola de Postman, procesando flujos encadenados de peticiones HTTP para certificar contratos de interfaz.
- **Capa 4 (End-to-End):** Implementada mediante **Playwright 1.52**, automatizando la interacción en navegadores reales para flujos críticos de la aplicación.

2.2. Configuración de Ambientes de Prueba Aislados

Para garantizar la reproducibilidad y el aislamiento de las pruebas (evitando la contaminación cruzada de datos), los entornos se diseñaron bajo esquemas efímeros e interceptación de infraestructura:

- **Ambiente Backend:** La persistencia de datos utiliza la librería `mongodb-memory-server`. Antes de iniciar cada suite de pruebas de Jest, se ejecuta un script de inicialización global que levanta una instancia real de MongoDB en la memoria RAM del sistema, aislada por completo de cualquier base de datos física o de red. Al concluir los tests, un script de desmontaje destruye la instancia liberando los recursos. Adicionalmente, la inyección de la variable de entorno `TEST_ENV=test` modifica el comportamiento del despachador de correos, interceptando los tokens de recuperación de contraseña y exponiéndolos en el cuerpo de la respuesta HTTP (`resetUrl`) para posibilitar su validación automatizada sin necesidad de servidores SMTP reales.
- **Ambiente Frontend:** Los tests unitarios de Vitest operan bajo el entorno de simulación de DOM `jsdom`, el cual expone las APIs del navegador directamente en el entorno de ejecución de Node.js. Por su parte, la capa E2E controlada por Playwright utiliza una estrategia de interceptación de tráfico mediante módulos ubicados en `e2e/helpers/apiMocks.js`. Estos scripts interceptan las llamadas salientes del frontend hacia el backend, simulando las respuestas JSON y los encabezados de sesión (`mockOwnerSession`), garantizando que la interfaz pueda probarse de forma aislada a la disponibilidad del servidor de base de datos.

2.3. Guía de Ejecución Técnica (Comandos de Consola y Prerrequisitos)

La ejecución del entorno de aseguramiento de calidad requiere Node.js instalado (versión igual o superior a 18) y el despliegue previo de las dependencias locales ejecutando el comando `npm install` o `npm ci` en la raíz de cada repositorio respectivo.

- **Comandos del Lado del Servidor (PetConnectBackend/):** `npm test` ejecuta la suite, `npm run test:coverage` genera el reporte de cobertura y `npm run test:smoke` corre las pruebas de humo.
- **Comandos del Lado del Cliente (PetConnect/):** `npm test` ejecuta los unitarios y `npm run test:e2e` arranca los escenarios de Playwright.

2.4. Ciclo de Vida de Integración Continuada (CI/CD) en GitHub Actions

El control de regresiones de PetConnect se encuentra completamente automatizado mediante dos flujos de trabajo independientes en GitHub Actions, configurados en los archivos `.github/workflows/backend-tests.yml` y `frontend-tests.yml` respectivamente. Cada vez que un desarrollador genera un evento de empuje (`push`) o una solicitud de extracción (`pull request`) hacia las ramas estables (`main` o `develop`), los servidores de integración de GitHub aprovisionan contenedores limpios sobre Ubuntu LTS, ejecutan la instalación automatizada de dependencias, levantan las bases de datos efímeras y ejecutan de manera obligatoria las suites correspondientes. El paso de producción o despliegue queda estrictamente bloqueado en el pipeline si una sola prueba de las suites automatizadas de Jest o Vitest reporta un estado de falla.

3. RESULTADOS CUANTITATIVOS Y CERTIFICACIÓN DE ENTORNOS

3.1. Cuadro Resumen de la Cobertura Técnica Automatizada

Los resultados obtenidos mediante la ejecución directa de las herramientas de prueba revelan el estado real de la calidad del software. El sistema cuenta con **194 pruebas automatizadas** en código fuente que garantizan la estabilidad de la aplicación frente a cambios regresivos:

Capa Técnica	Framework / Herramienta	Archivos Escaneados	Total Tests (Nº Real)	Estado Resultado /
Backend (Integración/Lógica)	Jest 29 + Supertest	17 archivos	150 tests	100% Exitosa (Pass)
Frontend (Unitario/Componentes)	Vitest 4 + Testing Library	14 archivos	38 tests	100% Exitosa (Pass)
Frontend E2E	Playwright 1.52	5 archivos	6 tests	Pasando (Vía CI)
Smoke Tests (Pre-deploy)	Newman CLI / Postman	1 Colección	13 Requests	17 Aserciones Exitosas

3.2. Auditoría y Conteo de Tests Automatizados Exitosos (150 Jest / 38 Vitest)

La auditoría de código realizada sobre los archivos de prueba arrojó la siguiente distribución de la cobertura técnica:

Distribución Física de Tests en Backend (Jest)

El servidor valida su comportamiento a través de 150 aserciones automatizadas distribuidas en los siguientes componentes de software esenciales:

- auth.routes.test.js (22) y profile.routes.test.js (4)
- pets.routes.test.js (28) y petAccess.service.test.js (6)
- appointments.routes.test.js (18), bookings.routes.test.js (4), providerAgenda.routes.test.js (4)
- providers.routes.test.js (11), clinicServices.routes.test.js (8), vetClinical.routes.test.js (11)
- admin.routes.test.js (11), medicalPdf.service.test.js (8), reviews.routes.test.js (5), chat.routes.test.js (2), jwt.test.js (5) y smoke.flow.test.js (1)

Distribución Física de Tests en Frontend (Vitest)

La interfaz del usuario se encuentra validada mediante 38 aserciones atómicas distribuidas en componentes clave:

- *Lógica y Datos (19 tests)*: Concentrados en el control de roles de usuario (userRoles.test.js), utilidades horarias chilenas frente al huso UTC (chileTime.test.js), manejo de peticiones (api.test.js) y lógica base de chat y utilidades.
- *Componentes Visuales y Estado (19 tests)*: Validan el comportamiento del contexto de autenticación (AuthProvider.test.jsx), formularios de Login y recuperación de claves, el widget del asistente inteligente (ChatWidget.test.jsx), el banner de pérdida de conectividad (OfflineBanner.test.jsx) y hooks personalizados.

3.3. Certificación del Entorno: Análisis del Flujo Smoke (SMK-001 → SMK-015)

El flujo de Smoke Testing o "Prueba de Humo" actúa como el guardián del despliegue, asegurando que los caminos más felices y esenciales de la API de PetConnect funcionen perfectamente en el servidor antes de autorizar cualquier tipo de prueba funcional masiva.

3.3.1. Orden Cronológico del Happy Path en la Colección Newman (13 Requests)

El script de Newman procesa de manera estrictamente secuencial las siguientes 13 peticiones encadenadas, emulando la interacción del negocio real:

1. **SMK-001 (Health Check)**: Llama a GET /health para certificar que el servidor responde.
2. **SMK-002 (Login Dueño)**: POST /api/auth/login para obtener un JWT válido.
3. **SMK-004 (Listar Mascotas)**: GET /api/pets para verificar la lectura de la base de datos.
4. **SMK-005 (Mapa Proveedores)**: GET /api/proveedores/mapa pasando coordenadas geográficas.
5. **SMK-006 (Buscar Veterinarias)**: GET /api/proveedores/buscar?tipo=veterinaria.
6. **SMK-007a (Slots Disponibles)**: Consulta las ventanas horarias libres de una clínica veterinaria.
7. **SMK-007b (Crear Cita)**: Despacha un POST /api/appointments consumiendo el identificador del slot obtenido en el paso anterior.
8. **SMK-008 (Bookings Dueño)**: Verifica que la cita aparezca en la bandeja del dueño de la mascota.
9. **SMK-009a (Login Veterinario)**: Inicia sesión con credenciales de personal médico.
10. **SMK-009 (Pacientes Vet)**: Llama a GET /api/vet/patients confirmando que la cita impactó la agenda del profesional.
11. **SMK-010 (Ficha Médica Resumen)**: GET /api/pets/:petId/medical-summary para validar la lectura del historial clínico de la mascota.
12. **SMK-011 (Chatbot)**: POST /api/chat enviando una consulta al motor de inteligencia artificial.
13. **SMK-015 (Forgot Password)**: POST /api/auth/forgot-password para corroborar el inicio del flujo de recuperación de cuentas.

3.3.2. Mapeo de Aserciones e Integridad del Flujo (17 Aserciones Newman)

La suite de Newman inyecta un total de **17 aserciones automáticas** escritas en bloques pm.test. El flujo se encuentra blindado gracias a que el paso 1 valida la

estructura JSON, el paso 2 extrae el token JWT de la respuesta y lo inyecta como variable global para las llamadas posteriores, y los pasos 6 y 7 evalúan que el arreglo de bloques horarios contenga elementos disponibles. Este mismo flujo completo está replicado de forma nativa en el backend en el archivo `smoke.flow.test.js`, corriendo de forma local en memoria con un único método de prueba que ejecuta las 17 aserciones mediante comandos `expect()` encadenados.

4. MATRIZ DETALLADA DE TRAZABILIDAD TÉCNICA PC-*

4.1. Estructura de Diseño y Campos de las Tablas de Casos de Prueba

Con el fin de asegurar el rigor metodológico exigido en auditorías de software, los casos de prueba mapeados a nivel de código se han organizado por módulos específicos, detallando su ID de trazabilidad, el componente técnico exacto bajo prueba, las precondiciones, el flujo de ejecución y el criterio de éxito verificado.

4.2. Mapeo Técnico de Casos Implementados por Módulos del Sistema

4.2.1. Cobertura en Módulo Autenticación y Autorización (AUTH / MID)

Este módulo se encuentra altamente automatizado en el archivo de integración `auth.routes.test.js` y las suites de componentes de login del cliente.

ID del Caso	Componente / Ruta Afectada	Descripción de la Prueba	Resultado Esperado en Código	Prioridad
PC-AUTH-001	POST /api/auth/register	Registro dueño exitoso.	HTTP 201. Retorna token JWT y objeto usuario con role: "dueno".	Alta
PC-AUTH-002	POST /api/auth/register	Intento de registro utilizando un correo electrónico duplicado.	HTTP 409 Conflict. Mensaje de error indica duplicidad.	Alta
PC-AUTH-003	POST /api/auth/login	Login exitoso de un usuario registrado aportando credenciales correctas.	HTTP 200 OK. Retorna el token de sesión y el objeto de perfil del usuario.	Alta
PC-AUTH-004	POST /api/auth/login	Intento de autenticación con contraseña incorrecta.	HTTP 400 Bad Request.	Alta
PC-AUTH-009	POST /api/auth/forgot-password	Solicitud de recuperación de contraseña	HTTP 200 OK. En entorno de test, expone la	Alta

ID del Caso	Componente / Ruta Afectada	Descripción de la Prueba	Resultado Esperado en Código	Prioridad
		ingresando un correo electrónico válido.	propiedad resetUrl con el token.	
PC-AUTH-010	POST /api/auth/reset-password	Actualización de contraseña utilizando el token de recuperación correcto.	HTTP 200 OK. Modifica el campo password en la base de datos.	Alta
PC-AUTH-012	GET /api/profile/me	Validación de rutas protegidas mediante el middleware de verificación de firmas.	HTTP 401 Unauthorized ante tokens inexistentes o expirados.	Alta
PC-PROF-001	GET /api/profile/me	Recuperación del perfil de usuario autenticado mediante sesión válida.	HTTP 200 OK. Retorna el payload del usuario logueado con éxito.	Media
PC-AUTH-005	Componente LoginPage	Renderizado del formulario y flujo completo de inicio de sesión en interfaz gráfica.	Llama a la función de navegación redirigiendo al usuario a la ruta raíz /.	Alta

4.2.2. Cobertura en Módulo Mascotas y Ficha Médica (PETS / MED / VET)

Módulo crítico que asegura la integridad de los datos de salud de los pacientes veterinarios.

ID del Caso	Componente / Ruta Afectada	Descripción de la Prueba	Resultado Esperado en Código	Prioridad
PC-PETS-001	POST /api/pets	Registro de una nueva mascota asociada al dueño autenticado.	HTTP 201 Created.	Alta

ID del Caso	Componente / Ruta Afectada	Descripción de la Prueba	Resultado Esperado en Código	Prioridad
PC-PETS-002	GET /api/pets	Recuperación de la lista de mascotas registradas por el dueño que realiza la consulta.	HTTP 200 OK. Retorna un arreglo JSON que incluye únicamente las mascotas del usuario.	Alta
PC-PETS-004	PATCH /api/pets/:id	Modificación de los datos parciales de una mascota.	HTTP 200 OK. Retorna el documento de la mascota actualizado.	Alta
PC-PETS-008	PATCH /api/pets/:id/deceased	Actualización del estado de vida de una mascota a fallecida.	HTTP 200 OK. Modifica el estado interno de la entidad a deceased.	Alta
PC-MED-001	GET /pets/:id/medical-summary	Consulta del resumen e historial clínico consolidado de una mascota específica.	HTTP 200 OK. Retorna el total de atenciones recibidas y un consolidado médico.	Alta
PC-MED-003	GET /pets/:id/encounters	Listado cronológico de las consultas o registros clínicos vinculados a la mascota.	HTTP 200 OK. Retorna un arreglo ordenado con las consultas históricas.	Alta
PC-MED-005	POST /api/vet/encounters	Creación de una ficha clínica por parte de un veterinario autorizado ligada a una cita.	HTTP 201 Created. Registra la consulta médica.	Alta
PC-MED-009	GET /export.pdf	Generación e importación del historial médico	HTTP 200 OK. El servidor responde con un flujo de	Alta

ID del Caso	Componente / Ruta Afectada	Descripción de la Prueba	Resultado Esperado en Código	Prioridad
		completo de la mascota en formato PDF.	bytes binarios de tipo application/pdf.	
PC-MED-012	petAccess.service.js	Validación lógica de derechos de acceso de un veterinario a la ficha de una mascota.	Retorna un valor booleano true únicamente si existe una cita confirmada vigente.	Alta

4.2.3. Cobertura en Módulo Citas y Reservas Unificadas (APPT / AGENDA / BOOK)

Este bloque técnico controla la lógica transaccional de separación de horas y atención de servicios.

ID del Caso	Componente / Ruta Afectada	Descripción de la Prueba	Resultado Esperado en Código	Prioridad
PC-APPT-001	GET /appointments/slots	Consulta de la grilla de bloques de tiempo disponibles para un proveedor médico.	HTTP 200 OK. Retorna un arreglo con las ventanas horarias libres.	Alta
PC-APPT-002	POST /api/appointments	Reserva de una cita médica consumiendo un bloque de tiempo disponible.	HTTP 201 Created. La cita queda registrada.	Alta
PC-APPT-004	POST /api/appointments	Intento de reserva de un bloque de tiempo que ya se encuentra ocupado.	HTTP 409 Conflict o 400 Bad Request. Bloquea la creación de la cita duplicada.	Alta

ID del Caso	Componente / Ruta Afectada	Descripción de la Prueba	Resultado Esperado en Código	Prioridad
PC-APPT-006	PATCH /appointments/:id/cancel	Cancelación de una reserva por parte del dueño.	HTTP 200 OK. Libera el slot de tiempo y actualiza el estado de la cita.	Alta
PC-APPT-007	PATCH /provider/confirm	Confirmación de la cita por parte del proveedor asignado.	HTTP 200 OK. Cambia el estado de la cita.	Alta
PC-AGENDA-002	GET /provider/agenda/slots	Extracción de los bloques de tiempo configurados por el propio profesional.	HTTP 200 OK. Retorna la grilla completa de la agenda del prestador.	Media
PC-AGENDA-003	PATCH /slots/:id/block	Bloqueo manual de una ventana de tiempo por razones operativas del proveedor.	HTTP 200 OK. El estado del bloque cambia directamente a blocked.	Media
PC-BOOK-001	GET /api/bookings/mine	Visualización unificada del historial de reservas del dueño de la mascota.	HTTP 200 OK. Retorna un objeto canónico con el consolidado de atenciones.	Alta

4.2.4. Cobertura en Módulos de Soporte (SEARCH / REV / CHAT / PROV / ADMIN)

Abarca el funcionamiento de la búsqueda, el motor de inteligencia artificial y la moderación del sistema.

ID del Caso	Componente / Ruta Afectada	Descripción de la Prueba	Resultado Esperado en Código	Prioridad
PC-SEARCH-001	GET /api/proveedores/buscar	Búsqueda parametrizada de	HTTP 200 OK. Retorna	Media

ID del Caso	Componente / Ruta Afectada	Descripción de la Prueba	Resultado Esperado en Código	Prioridad
		centros veterinarios activos por tipo de servicio.	un listado estructurado de prestadores.	
PC-SEARCH-003	GET /api/proveedores/mapa	Carga de geolocalización para renderizar marcadores en el mapa del cliente.	HTTP 200 OK. Retorna las coordenadas e información básica de los puntos cercanos.	Media
PC-REV-002	POST /api/reviews	Publicación de una calificación y reseña una vez completada una atención.	HTTP 201 Created. Almacena la puntuación.	Media
PC-REV-004	POST /reviews/:id/report	Reporte de una reseña por parte de un prestador debido a lenguaje inapropiado.	HTTP 201 Created. Levanta una bandera de auditoría.	Media
PC-CHAT-001	POST /api/chat	Envío de una consulta médica simulada al asistente inteligente Vetto.	HTTP 200 OK. Retorna una respuesta textual simulada del modelo de IA.	Media
PC-PROV-004	POST /provider/clinic-services	Inserción de un nuevo servicio clínico en el catálogo de la veterinaria aprobada.	HTTP 201 Created. El servicio se añade al catálogo base del prestador.	Media

ID del Caso	Componente / Ruta Afectada	Descripción de la Prueba	Resultado Esperado en Código	Prioridad
PC-ADMIN-001	GET /admin/providers/pending	Recuperación de la lista de prestadores en espera de validación de credenciales.	HTTP 200 OK. Retorna un arreglo con las solicitudes de registro pendientes.	Media
PC-ADMIN-002	PATCH /admin/providers/:id/approve	Aprobación de una cuenta de proveedor por parte del rol administrador.	HTTP 200 OK. Cambia el estado a aprobado.	Media

4.2.5. Cobertura de Robustez y Variantes Negativas (-NEG en Código)

Los archivos de pruebas de integración no solo validan los flujos ideales de éxito, sino que contienen aserciones complementarias diseñadas para garantizar la resiliencia de la API ante ataques o peticiones malformadas:

- *Manejo de Errores de Entrada (400 Bad Request)*: Validado en registros sin contraseñas, intentos de inyección de parámetros horarios erróneos en el agendamiento y creación de mascotas con especies no soportadas por las reglas del negocio (dragon).
- *Protección y Privacidad Cruzada (403 Forbidden)*: Validado mediante pruebas donde un veterinario intenta consultar de forma directa el historial clínico de una mascota con la cual no posee ningún tipo de cita activa confirmada en el sistema, asegurando el bloqueo absoluto de la información privada.

5. ANÁLISIS DE BRECHAS Y TRAZABILIDAD EXTERNA

5.1. Evaluación del Porcentaje Real de Cobertura de la Matriz (Cálculo del ~59% Real)

La auditoría automatizada realizada sobre el repositorio de código revela una **desconexión del 41% entre la suite de pruebas teórica planificada y los scripts automatizados reales**. De los 105 casos de prueba formalizados en el documento de planeación estratégica (TEST_PLAN.md), **únicamente 62 casos cuentan con una cobertura automatizada ejecutable directa o parcial** en el código de Jest, Vitest o Playwright. Los 43 casos restantes se mantienen actualmente bajo modalidad de aseguramiento manual, dependiendo enteramente de la ejecución guiada en navegadores mediante checklists tradicionales.

5.2. Inventario de Casos Críticos de Prioridad Alta Sin Automatizar

El análisis detectó la ausencia de scripts de automatización para varios flujos tipificados como **Prioridad Alta (Críticos)** dentro de la estrategia de negocio:

1. **Flujo de Cierre de Sesión (PC-AUTH-006)**: No existen aserciones automatizadas de integración ni de interfaz que verifiquen la invalidación del token o la limpieza del almacenamiento local (localStorage) al efectuar un logout.
2. **Registro de Proveedores Médicos (PC-AUTH-007 / 008)**: El flujo de registro para clínicas y paseadores está completamente desprovisto de cobertura automática a nivel de rutas, delegando toda su validación a las pruebas manuales.
3. **Interfaces de Creación y Edición Gráfica (Módulos UI)**: Los formularios de creación de mascotas (PC-PETS-005), agendamiento visual de horas (PC-APPT-003) y visualización de la cartola de reservas (PC-BOOK-002) no poseen pruebas funcionales con Testing Library ni Playwright, existiendo únicamente cobertura en la capa lógica de las llamadas de la API.

5.3. Discrepancias entre Especificaciones del Plan Teórico y Código Fuente

Se identificó una inconsistencia de integración entre el diseño lógico de QA y el desarrollo técnico:

- *El Contrato de Error en Login (PC-AUTH-004)*: Mientras que el plan de pruebas estipula formalmente que el servidor debe responder con un código de estado 401 Unauthorized ante intentos de inicio de sesión con credenciales inválidas, el código real emite un 400 Bad Request. Las pruebas de integración de Jest fueron escritas esperando el código 400, lo que valida la estabilidad del backend actual pero evidencia una falta de consistencia con las especificaciones del plan original.

5.4. Estado de la Trazabilidad y Sincronización con Herramientas (Jira Test / Xray)

Al revisar la infraestructura de integración del proyecto, **se certifica que no existe ningún tipo de enlace automatizado hacia Jira ni herramientas de Test Management como Xray o Zephyr**. La trazabilidad del proyecto se encuentra en un estado de **aislamiento de herramientas**: los criterios de aceptación residen de forma estática en la exportación de Excel, los casos de prueba están declarados en archivos Markdown (TEST_PLAN.md) y los resultados de éxito se limitan a la consola local o los registros planos del pipeline de GitHub Actions, sin integrarse directamente en los tickets de Jira correspondientes.

6. GESTIÓN DE RIESGOS EN EL PROCESO DE PRUEBAS

6.1. Criterios de Entrada para el Ciclo de QA

Para autorizar el inicio del proceso de pruebas en cualquier fase de desarrollo de PetConnect, se deben cumplir obligatoriamente las siguientes condiciones de entrada:

1. El código fuente de la funcionalidad debe estar integrado en la rama de desarrollo correspondiente (develop o main) sin presentar conflictos de mezcla.
2. El entorno de ejecución de Node.js debe estar configurado de manera homogénea en su versión estable de soporte a largo plazo (LTS v18 o superior).
3. Las dependencias de desarrollo (devDependencies), incluyendo los motores de bases de datos en memoria y los binarios de los navegadores de Playwright, deben estar correctamente instalados y compilados.

6.2. Criterios de Out / Salida para la Aprobación del Release

Una versión del sistema PetConnect se considera aprobada y certificada para pasar a producción únicamente cuando cumple con los siguientes umbrales de salida:

1. El **100% de las pruebas automatizadas existentes (188 tests entre Jest y Vitest) deben reportar un estado exitoso (Pass)**; una sola falla detiene el despliegue.
2. La cobertura de ramas de código (branch coverage) en el backend debe ser igual o superior al **70%** como métrica de calidad estructural.
3. El flujo de humo automatizado en Newman (carpeta Smoke) debe completarse con **cero errores** sobre el servidor de pruebas o Staging.

6.3. Matriz de Riesgos Técnicos e Identificación de Mitigaciones

Durante el proceso de auditoría y ejecución de la suite de control de calidad, se identificaron los siguientes riesgos técnicos y sus correspondientes estrategias de mitigación implementadas en el código:

- **Riesgo 1: Dependencia de Servicios Externos de IA (OpenAI API):** Las pruebas del chatbot corrían el riesgo de fallar o generar cobros excesivos por llamadas repetitivas si la API externa presentaba latencia o problemas de autenticación.
 - *Mitigación:* Se implementó un aislamiento completo inyectando un mock global sobre el proveedor del modelo de lenguaje para simular respuestas estáticas de manera inmediata sin realizar llamadas reales hacia la red.
- **Riesgo 2: Fallas por Desfase de Fechas en Agendamiento de Turnos:** Las pruebas automatizadas de reserva de citas corrían el riesgo de fallar por el paso del tiempo si utilizaban fechas estáticas fijas en el código, ya que los bloques de agenda expirarían continuamente.

- *Mitigación:* Se configuró la función `createAvailableSlot` para que calcule de manera dinámica los días utilizando la utilidad horaria del sistema, garantizando que los bloques de tiempo generados pertenezcan siempre al día en curso o al día de mañana.

7. CONCLUSIONES

7.1. Dictamen de Calidad sobre el Sprint 6

El veredicto técnico determina que **el sistema PetConnect es aprobable con observaciones menores para su salida a producción**. Si bien existe una brecha importante en la automatización de la interfaz del usuario, la capa de servicios lógicos, el motor de autenticación, la consistencia de la base de datos y las reglas del negocio de agendamiento se encuentran fuertemente respaldadas por **150 pruebas de integración exitosas**. Esto garantiza que el núcleo del sistema es altamente estable y resiliente ante errores graves.

7.2. Lecciones Aprendidas de la Estrategia de Testing

La gran lección del proceso de QA de PetConnect fue comprobar la tremenda utilidad de haber definido los Criterios de Aceptación desde el inicio del backlog. Esto permitió al equipo construir una suite robusta de pruebas automatizadas que atrapa regresiones lógicas de inmediato. No obstante, el proceso evidenció el peligro de mantener la documentación aislada del código; los planes teóricos envejecen muy rápido si las aserciones reales del software cambian durante la etapa de programación.

7.3. Plan de Acción y Próximos Pasos de QA para Futuros Sprints

Con el fin de subsanar las brechas de aseguramiento detectadas en esta auditoría, se establece el siguiente plan de acción prioritario para los próximos ciclos de trabajo:

1. **Completar la Cobertura E2E (Prioridad Alta):** Escribir e integrar en Playwright los flujos reales de la interfaz de usuario para el inicio de sesión, el cierre de sesión, la creación de mascotas y la reserva de turnos, eliminando la dependencia de los mocks de red actuales.
2. **Unificar Contratos de Interfaz:** Ajustar el código del backend o rectificar el plan escrito para que el error de credenciales inválidas guarde consistencia absoluta entre lo que dice el documento y lo que responde el servidor.
3. **Automatizar la Trazabilidad con Jira:** Configurar scripts en el pipeline de GitHub Actions para que los resultados de las pruebas impacten y actualicen de forma automática las tareas del backlog de Excel en Jira, eliminando el traspaso manual de información.